

HTML、XML、XHTML 的区别

- HTML：超文本标记语言，是语法较为松散的、不严格的Web语言；
- XML：可扩展的标记语言，主要用于存储数据和结构，可扩展；
- XHTML：可扩展的超文本标记语言，基于XML，作用与HTML类似，但语法更严格。

HTML5有哪些更新

1. 语义化标签

- header：定义文档的页眉（头部）；
- nav：定义导航链接的部分；
- footer：定义文档或节的页脚（底部）；
- article：定义文章内容；
- section：定义文档中的节（section、区段）；
- aside：定义其所处内容之外的内容（侧边）；

2. 媒体标签

(1) audio：音频

```
<audio src='' controls autoplay loop='true'></audio>
```

属性：

- controls 控制面板
- autoplay 自动播放
- loop='true' 循环播放

(2) video视频

```
<video src='' poster='imgs/aa.jpg' controls></video>
```

属性：

- poster：指定视频还没有完全下载完毕，或者用户还没有点击播放前显示的封面。默认显示当前视频文件的第一帧画面，当然通过poster也可以自己指定。
- controls 控制面板
- width
- height

(3) source标签 因为浏览器对视频格式支持程度不一样，为了能够兼容不同的浏览器，可以通过source来指定视频源。

```
<video>
  <source src='aa.flv' type='video/flv'></source>
  <source src='aa.mp4' type='video/mp4'></source>
</video>
```

3. 表单

表单类型：

- email：能够验证当前输入的邮箱地址是否合法
- url：验证URL
- number：只能输入数字，其他输入不了，而且自带上下增大减小箭头，max属性可以设置为最大值，min可以设置为最小值，value为默认值。
- search：输入框后面会给提供一个小叉，可以删除输入的内容，更加人性化。
- range：可以提供给一个范围，其中可以设置max和min以及value，其中value属性可以设置为默认值
- color：提供了一个颜色拾取器
- time：时分秒
- data：日期选择年月日
- datetime：时间和日期(目前只有Safari支持)
- datetime-local：日期时间控件
- week：周控件
- month：月控件

表单属性：

- placeholder：提示信息
- autofocus：自动获取焦点
- autocomplete="on" 或者 autocomplete="off" 使用这个属性需要有两个前提：
 - 表单必须提交过
 - 必须有name属性。
- required：要求输入框不能为空，必须有值才能够提交。
- pattern=" " 里面写入想要的正则模式，例如手机号patte="^(+86)?\d{10}\$"
- multiple：可以选择多个文件或者多个邮箱
- form=" form表单的ID"

表单事件：

- oninput 每当input里的输入框内容发生变化都会触发此事件。
- oninvalid 当验证不通过时触发此事件。

4. 进度条、度量器

- progress标签：用来表示任务的进度（IE、Safari不支持），max用来表示任务的进度，value表示已完成多少
- meter属性：用来显示剩余容量或剩余库存（IE、Safari不支持）
 - high/low：规定被视作高/低的范围
 - max/min：规定最大/小值
 - value：规定当前度量值

设置规则：min < low < high < max

5. DOM查询操作

- document.querySelector()
- document.querySelectorAll()

它们选择的对象可以是标签，可以是类(需要加点)，可以是ID(需要加#)

6. Web存储

HTML5 提供了两种在客户端存储数据的新方法：

- localStorage - 没有时间限制的数据存储
- sessionStorage - 针对一个 session 的数据存储

7. 其他

- 拖放：拖放是一种常见的特性，即抓取对象以后拖到另一个位置。设置元素可拖放：

```
<img draggable="true" />
```

- 画布（canvas）： canvas 元素使用 JavaScript 在网页上绘制图像。画布是一个矩形区域，可以控制其每一像素。canvas 拥有多种绘制路径、矩形、圆形、字符以及添加图像的方法。

```
<canvas id="myCanvas" width="200" height="100"></canvas>
```

- SVG：SVG 指可伸缩矢量图形，用于定义用于网络的基于矢量的图形，使用 XML 格式定义图形，图像在放大或改变尺寸的情况下其图形质量不会有损失，它是万维网联盟的标准
- 地理定位：Geolocation（地理定位）用于定位用户的位置。‘

总结：

- (1) 新增语义化标签：nav、header、footer、aside、section、article
- (2) 音频、视频标签：audio、video
- (3) 数据存储：localStorage、sessionStorage
- (4) canvas（画布）、Geolocation（地理定位）、websocket（通信协议）
- (5) input标签新增属性：placeholder、autocomplete、autofocus、required
- (6) history API：go、forward、back、pushstate

移除的元素有：

- 纯表现的元素：basefont, big, center, font, s, strike, tt, u;
- 对可用性产生负面影响的元素：frame, frameset, noframes;

常见的meta标签有哪些

meta 标签由 name 和 content 属性定义，**用来描述网页文档的属性**，比如网页的作者，网页描述，关键词等，除了HTTP标准固定了一些 name 作为大家使用的共识，开发者还可以自定义name。

常见的meta标签：

- (1) charset，用来描述HTML文档的编码类型：

```
<meta charset="UTF-8" >
```

- (2) keywords，页面关键词：

```
<meta name="keywords" content="关键词" />
```

- (3) description，页面描述：

```
<meta name="description" content="页面描述内容" />
```

- (4) refresh，页面重定向和刷新：

```
<meta http-equiv="refresh" content="0;url=" />
```

(5) `viewport`，适配移动端，可以控制视口的大小和比例：

```
<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
```

其中，`content` 参数有以下几种：

- `width viewport`：宽度(数值/device-width)
- `height viewport`：高度(数值/device-height)
- `initial-scale`：初始缩放比例
- `maximum-scale`：最大缩放比例
- `minimum-scale`：最小缩放比例
- `user-scalable`：是否允许用户缩放(yes/no)

(6) 搜索引擎索引方式：

```
<meta name="robots" content="index,follow" />
```

其中，`content` 参数有以下几种：

- `all`：文件将被检索，且页面上的链接可以被查询；
- `none`：文件将不被检索，且页面上的链接不可以被查询；
- `index`：文件将被检索；
- `follow`：页面上的链接可以被查询；
- `noindex`：文件将不被检索；
- `nofollow`：页面上的链接不可以被查询。

行内元素有哪些？块级元素有哪些？空(void)元素有那些？

- 行内元素有：`a b span img input select strong`；
- 块级元素有：`div ul ol li dl dt dd h1 h2 h3 h4 h5 h6 p`；

空元素，即没有内容的HTML元素。空元素是在开始标签中关闭的，也就是空元素没有闭合标签：

- 常见的有：`<br>`、`<hr>`、`<img>`、`<input>`、`<link>`、`<meta>`；
- 罕见的有：`<area>`、`<base>`、`<col>`、`<colgroup>`、`<command>`、`<embed>`、`<keygen>`、`<param>`、`<source>`、`<track>`、`<wbr>`。

说一下 HTML5 drag API

- `dragstart`：事件主体是被拖放元素，在开始拖放被拖放元素时触发。
- `darg`：事件主体是被拖放元素，在正在拖放被拖放元素时触发。
- `dragenter`：事件主体是目标元素，在被拖放元素进入某元素时触发。
- `dragover`：事件主体是目标元素，在被拖放在某元素内移动时触发。
- `dragleave`：事件主体是目标元素，在被拖放元素移出目标元素是触发。
- `drop`：事件主体是目标元素，在目标元素完全接受被拖放元素时触发。
- `dragend`：事件主体是被拖放元素，在整个拖放操作结束时触发。

HTML公共属性/全局属性

属性	描述
accesskey	规定激活元素的快捷键。
class	规定元素的一个或多个类名（引用样式表中的类）。
contenteditable	规定元素内容是否可编辑。
contextmenu	规定元素的上下文菜单。上下文菜单在用户点击元素时显示。
data-*	用于存储页面或应用程序的私有定制数据。
dir	规定元素中内容的文本方向。
draggable	规定元素是否可拖动。
dropzone	规定在拖动被拖动数据时是否进行复制、移动或链接。
hidden	规定元素仍未或不再相关。
id	规定元素的唯一 id。
lang	规定元素内容的语言。
spellcheck	规定是否对元素进行拼写和语法检查。
style	规定元素的行内 CSS 样式。
tabindex	规定元素的 tab 键次序。
title	规定有关元素的额外信息。
translate	规定是否应该翻译元素内容。

HTML自定义属性

- 使用 data-* 属性来嵌入自定义数据：

```
<ul>
  <li data-animal-type="鸟类">喜鹊</li>
  <li data-animal-type="鱼类">金枪鱼</li>
  <li data-animal-type="蜘蛛">蝇虎</li>
</ul>
```

定义和用法

- data-* 属性用于存储页面或应用程序的私有自定义数据。
- data-* 属性赋予我们在所有 HTML 元素上嵌入自定义 data 属性的能力。
- 存储的（自定义）数据能够被页面的 JavaScript 中利用，以创建更好的用户体验（不进行 Ajax 调用或服务端数据库查询）。
- data-* 属性包括两部分：
 - 属性名不应该包含任何大写字母，并且在前缀 "data-" 之后必须有至少一个字符
 - 属性值可以是任意字符串

注释：用户代理会完全忽略前缀为 "data-" 的自定义属性。

data-* 属性是 HTML5 中的新属性。

```
<element data-*="somevalue">
```

(牛客)img的title属性和alt属性有什么区别？

- alt 用于图片无法加载时显示 Title 为该属性提供信息，通常当鼠标滑动到元素上的时候显示

渐进增强和优雅降级之间的区别

- 渐进增强：针对低版本浏览器进行构建页面，保证最基本的功能，然后再针对高级浏览器进行效果、交互等改进和追加功能达到更好的用户体验。
- 优雅降级：一开始就构建完整的功能，然后再针对低版本浏览器进行兼容。
- 区别：优雅降级是从复杂的现状开始，并试图减少用户体验的供给，而渐进增强则是从一个非常基础的，能够起作用的版本开始，并不断扩充，以适应未来环境的需要。
 - “优雅降级”观点认为应该针对那些最高级、最完善的浏览器来设计网站。而将那些被认为“过时”或有功能缺失的浏览器下的测试工作安排在开发周期的最后阶段，并把测试对象限定为主流浏览器（如 IE、Mozilla 等）的前一个版本。在这种设计范例下，旧版的浏览器被认为仅能提供“简陋却无妨”的浏览体验。你可以做一些小的调整来适应某个特定的浏览器。但由于它们并非我们所关注的焦点，因此除了修复较大的错误之外，其它的差异将被直接忽略。
 - “渐进增强”观点则认为应关注于内容本身。内容是我们建立网站的诱因。有的网站展示它，有的则收集它，有的寻求，有的操作，还有的网站甚至会包含以上的种种，但相同点是它们全都涉及到内容。这使得“渐进增强”成为一种更为合理的设计范例。这也是它立即被 Yahoo! 所采纳并用以构建其“分级式浏览器支持”策略的原因所在。

DOCTYPE(文档类型) 的作用

DOCTYPE是HTML5中一种标准通用标记语言的文档类型声明，它的目的是告诉浏览器（解析器）应该以什么样（html或xhtml）的文档类型定义来解析文档，不同的渲染模式会影响浏览器对 CSS 代码甚至 JavaScript 脚本的解析。它必须声明在HTML文档的第一行。

浏览器渲染页面的两种模式：

- **CSS1Compat: 标准模式（默认模式）**，浏览器使用W3C的标准解析渲染页面。在标准模式中，浏览器以其支持的最高标准呈现页面。
- **BackCompat: 怪异模式(混杂模式)**，浏览器使用自己的怪异模式解析渲染页面。在怪异模式中，页面以一种比较宽松的向后兼容的方式显示。

怪异(Quirks)模式和标准(Standards)模式有什么区别？

所谓的标准模式是指，浏览器按W3C标准解析执行代码；怪异模式则是使用浏览器自己的方式解析执行代码，因为不同浏览器解析执行的方式不一样，所以我们称之为怪异模式。浏览器解析时到底使用标准模式还是怪异模式，与你网页中的DTD声明直接相关，DTD声明定义了标准文档的类型（标准模式解析）文档类型，会使浏览器使用相应的方式加载网页并显示，忽略DTD声明,将使网页进入怪异模式(quirks mode)。(摘自牛客)

- **盒模型**：在W3C标准中，如果设置一个元素的宽度和高度，指的是元素内容的宽度和高度，而在Quirks 模式下，IE的宽度和高度还包含了padding和border。

- **设置行内元素的高宽：** 在Standards模式下，给等行内元素设置width和height都不会生效，而在quirks模式下，则会生效。
- **设置百分比的高度：** 在standards模式下，一个元素的高度是由其包含的内容来决定的，如果父元素没有设置百分比的高度，子元素设置一个百分比的高度是无效的;使用margin:0 auto在standards模式下可以使元素水平居中，但在quirks模式下却会失效。

(其他) Doctype文档类型有几种

注意区分上题

标签可声明三种 DTD 类型，分别表示**严格版本**、**过渡版本**以及**基于框架的 HTML 文档**。

- HTML 4.01 规定了三种文档类型：分别是 Strict、Transitional 以及 Frameset；
- XHTML 1.0 规定了三种 XML 文档类型：分别是 Strict、Transitional 以及 Frameset；
- Standards（标准）模式（也就是严格呈现模式）用于呈现遵循最新标准的网页；
- Quirks（包容）模式（也就是松散呈现模式或者兼容模式）用于呈现为传统浏览器而设计的网页。

新的HTML5文档类型和字符集是什么

- HTML5 文档类型：<!doctype html>
- 字符集：HTML5 使用的编码 <meta charset="UTF-8">

HTML语义化

什么是 HTML 语义化？

- 根据内容的结构化（内容语义化），选择合适的标签（代码语义化）便于开发者阅读和写出更优雅的代码的同时让浏览器的爬虫和机器很好地解析。

为什么要语义化？

- 为了在没有CSS的情况下，页面也能呈现出很好地内容结构、代码结构：为了裸奔时好看；
- 用户体验：例如title、alt 用于解释名词或解释图片信息、label 标签的活用；
- 有利于SEO：和搜索引擎建立良好沟通，有助于爬虫抓取更多的有效信息：爬虫依赖于标签来确定上下文和各个关键字的权重；
- 方便其他设备解析（如屏幕阅读器、盲人阅读器、移动设备）以意义的方式来渲染网页；
- 便于团队开发和维护，语义化更具可读性，是下一步网页的重要动向，遵循W3C标准的团队都遵循这个标准，可以减少差异化。

Canvas和SVG的区别

(1) SVG：SVG可缩放矢量图形是基于可扩展标记语言XML描述的语言，SVG基于XML就意味着SVG DOM中的每个元素都是可用的，可以为某个元素附加javascript事件处理器。在 SVG 中，每个被绘制的图形均被视为对象。如果 SVG 对象的属性发生变化，那么浏览器能够自动重现图形。

其特点如下：

- 不依赖分辨率
- 支持事件处理器
- 最适合带有大型渲染区域的应用程序
- 复杂度高会减慢渲染速度
- 不适合游戏应用

(2) Canvas: Canvas是画布, 通过JavaScript来绘制2D图形, 是逐像素进行渲染的。其位置发生改变, 就会重新进行绘制。

其特点如下:

- 依赖分辨率
- 不支持事件处理器
- 弱的文本渲染能力
- 能够以 .png 或 .jpg 格式保存结果图像
- 最适合图像密集型的游戏, 其中的许多对象会被频繁重绘

注: 矢量图, 也称为面向对象的图像或绘图图像, 在数学上定义为一系列由线连接的点。矢量文件中的图形元素称为对象。每个对象都是一个自成一体的实体, 它具有颜色、形状、轮廓、大小和屏幕位置等属性。

单页应用和多页应用

单页应用

- 每一次页面跳转, 不会出现新的HTML文档, 但页面会跟着改变。原因是JavaScript会感知到页面UIL的变化, 感知到变化之后, 将当前页面的内容清除, 然后挂载上下一个页面的内容。也就是说, 单页应用是通过JS渲染来跳转页面。
- 优点: 页面切换快, 不需要加载HTML文件, 减少http请求、发送的时延。
- 缺点: 首屏时间稍慢, 因为需要发送一个http请求和js请求, 当两个请求都返回时, 首屏才可以展示; SEO效果差。

多页应用

- 每一次页面的跳转, 后台服务器都会返回一个新的HTML文档, 这样的网站我们称之为多页网站, 也叫多页面应用。简单来说, 多页应用是通过http请求来跳转页面。
- 优点: 首屏时间快, 即页面首个页面展现出来的时间短; SEO效果好。
- 缺点: 页面切换慢, 因为每一次跳转页面都要发送一次http请求, 假设网络非常慢, 则跳转的时候会出现卡顿的情况。

对比项 \ 模式	SPA	MPA
结构	一个主页面 + 许多模块的组件	许多完整的页面
体验	页面切换快，体验佳；当初次加载文件过多时，需要做相关的调优。	页面切换慢，网速慢的时候，体验尤其不好
资源文件	组件公用的资源只需要加载一次	每个页面都要自己加载公用的资源
适用场景	对体验度和流畅度有较高要求的应用，不利于 SEO（可借助 SSR 优化 SEO）	适用于对 SEO 要求较高的应用
过渡动画	Vue 提供了 transition 的封装组件，容易实现	很难实现
内容更新	相关组件的切换，即局部更新	整体 HTML 的切换，费钱（重复 HTTP 请求）
路由模式	可以使用 hash，也可以使用 history	普通链接跳转
数据传递	因为单页面，使用全局变量就好（Vuex）	cookie、localStorage 等缓存方案，URL 参数，调用接口保存等
相关成本	前期开发成本较高，后期维护较为容易	前期开发成本低，后期维护就比较麻烦，因为可能一个功能需要改很多地方

牛客@AprilEcho

link和@import的区别

- 区别1：link 是 XHTML 标签，除了加载 CSS 外，还可以定义 RSS 等其他事务；@import 属于 CSS 范畴，只能加载 CSS。
- 区别2：link 引用 CSS 时，在页面载入时同时加载；@import 需要页面网页完全载入以后加载。
- 区别3：link 是 XHTML 标签，无兼容问题；@import 是在 CSS2.1 提出的，低版本的浏览器不支持。
- 区别4：link 支持使用 Javascript 控制 DOM 去改变样式；而 @import 不支持。

超链接target属性的取值和作用？

- target这个属性指定所链接的页面在浏览器窗口中的打开方式。

它的参数值主要有：

- _blank：在新浏览器窗口中打开链接文件
- _parent：将链接的文件载入含有该链接框架的父框架集或父窗口中。如果含有该链接的框架不是嵌套的，则在浏览器全屏窗口中载入链接的文件，就像_self 参数一样。

- `_self`：在同一框架或窗口中打开所链接的文档。此参数为默认值，通常不用指定。
- `_top`：在当前的整个浏览器窗口中打开所链接的文档，因而会删除所有框架。

iframe优缺点

优点

- `iframe`能够原封不动的把嵌入的网页展现出来。
- 如果有多个网页引用`iframe`，那么只需要修改`iframe`的内容，就可以实现调用每一个页面的更改，方便快捷。
- 网页如果为了统一风格，头部和版本都是一样的，就可以写成一个页面，用`iframe`嵌套，可以增加代码的可重用。
- 如果遇到加载缓慢的第三方内容，如图标或广告，这些问题可以由`iframe`来解决。

缺点

- `iframe`会阻塞主页面的 `Onload` 事件；
- 搜索引擎的检索程序无法解读这种页面，不利于 `SEO`；
- `iframe`和主页面共享连接池，而浏览器对相同域的连接有限制，所以会影响页面的并行加载。
- 使用`iframe`之前需要考虑这两个缺点。如果需要使用 `iframe`，最好是通过 `javascript`动态给 `iframe`添加 `src` 属性值，这样可以避开以上两个问题。

src与href的区别

`src`和`href`都是用来引用外部的资源，它们的区别如下：

- **src**：表示对资源的引用，它指向的内容会嵌入到当前标签所在的位置。`src`会将其指向的资源下载并应用到文档内，如请求`js`脚本。当浏览器解析到该元素时，会暂停其他资源的下载和处理，直到将该资源加载、编译、执行完毕，所以一般`js`脚本会放在页面底部。
- **href**：表示超文本引用，它指向一些网络资源，建立和当前元素或本文档的链接关系。当浏览器识别到它他指向的文件时，就会并行下载资源，不会停止对当前文档的处理。常用在`a`、`link`等标签上。

title与h1的区别、b与strong的区别、i与em的区别

- `strong`标签有语义，是起到加重语气的效果，而`b`标签是没有的，`b`标签只是一个简单加粗标签。`b`标签之间的字符都设为粗体，`strong`标签加强字符的语气都是通过粗体来实现的，而搜索引擎更侧重`strong`标签。
- `title`属性没有明确意义只表示是个标题，`H1`则表示层次明确的标题，对页面信息的抓取有很大的影响
- `i`内容展示为斜体，`em`表示强调的文本

label标签的作用

- `label`标签来定义表单控制间的关系,当用户选择该标签时，浏览器会自动将焦点转到和标签相关的表单控件上。

```
<label for="Name">Number:</label>
<input type='text' name="Name" id="Name"/>
<label>Date:<input type="text" name="B"/></label>
```

html 中 title 属性和 alt 属性的区别

```

```

当图片不输出信息的时候，会显示 alt 信息 鼠标放上去没有信息，当图片正常读取，不会出现 alt 信息。

```

```

- 当图片不输出信息的时候，会显示 alt 信息 鼠标放上去会出现 title 信息；
- 当图片正常输出的时候，不会出现 alt 信息，鼠标放上去会出现 title 信息。
- 除了纯装饰图片外都必须设置有意义的值，搜索引擎会分析。

主流浏览器机器内核

浏览器	内核	备注
IE	Trident	IE、猎豹安全、360 极速浏览器、 百度 浏览器
firefox	Gecko	-
Safari	webkit	从 Safari 推出之时起，它的渲染引擎就是 Webkit，一提到 webkit，首先想到的便是 chrome，WebKit 的鼻祖其实是 Safari。
chrome	Chromium/Blink	在 Chromium 项目中研发 Blink 渲染引擎（即浏览器核心），内置于 Chrome 浏览器之中。Blink 其实是 WebKit 的分支。大部分国产浏览器最新版都采用 Blink 内核。二次开发
Opera	blink	Opera 内核原为：Presto，现在跟随 chrome 用 blink 内核。

（牛客）浏览器页面有哪三层构成，分别是什么，作用是什么？

- 构成：结构层、表示层、行为层
- 分别是：HTML、CSS、JavaScript
- 作用：HTML实现页面结构，CSS完成页面的表现与风格，JavaScript实现一些客户端的功能与业务。

（其他）浏览器乱码的原因是什么？如何解决？

产生乱码的原因

- 网页源代码是 gbk 的编码，而内容中的中文字是 utf-8 编码的，这样浏览器打开即会出现 html 乱码。反之也会出现乱码；
- html 网页编码是 gbk，而程序从数据库中调出呈现是 utf-8 编码的内容也会造成编码乱码；
- 浏览器不能自动检测网页编码，造成网页乱码。

解决办法

- 使用软件进行编辑 HTML 网页内容；
- 如果网页设置编码是 gbk，而数据库储存数据编码格式是 UTF-8，此时需要程序查询数据库数据显示数据前进行程序转码；
- 如果浏览器浏览时候出现网页乱码，在浏览器中找到转换编码的菜单进行转换；

说一下 web worker

在 HTML 页面中，如果在执行脚本时，页面的状态是不可相应的，直到脚本执行完成后，页面才变成可相应。web worker 是运行在后台的 js，独立于其他脚本，不会影响页面的性能。并且通过 postMessage 将结果回传到主线程。这样在进行复杂操作的时候，就不会阻塞主线程了。

如何创建 web worker：

1. 检测浏览器对于 web worker 的支持性
2. 创建 web worker 文件（js，回传函数等）
3. 创建 web worker 对象